淺度機器學習分類器比較 -- 使用資料 wine，並將其設定為ds0

學號：411079060

姓名：石欽鈺

完成日期：2023/5/12

作品目標： 在這個數據龐大的時代，多變量、大數據、即時反應已是很常見的場景。此時，機器學習是解決這些需求的工具，而我們想知道不同的分類器在不同資料面前的性能，於是進行 評比實驗 來探討機器學習中常見的分類方法，包括 多元羅吉斯回歸（`MLR`）、支援向量機（`SVM`）、以及神經網路（`Neural Network`），同時我們也想知道用原始資料與主成分資料於分類時的差異，無論是準確度的變化還是整體時間耗費多寡。因此，我們還會將資料分成原始資料與主成分資料，除此之外，部分資料還有做看看降噪處理，並去觀察他的性能：

1. 介紹並認識三種常見的統計分類方法：多元羅吉斯回歸、支援向量機、神經網路。
2. 評比實驗：
   - 使用三種不同的資料集，包括大小、變數多寡、資料型態等不同特徵的資料，來測試三種分類器的準確度。
   - 準確度的優化: 包括但不限於參數的調整（如正規化、gamma、degree等）、計算方式的選擇（如演算法、激活函數等）、以及資料預處理（如標準化、PCA / SVD、降噪等）。
3. 針對不同資料： 原始 & 取前5個主成分 ，找出最佳準確度組合，並去比較後哪種分類器在影像分類任務中表現最佳， 不同資料型態在分類時會產生什麼樣的差異 ，並給出相應的建議和結論。 </font>

羅吉斯迴歸（Logistic Regression）模型以及貝氏定理的應用

羅吉斯迴歸（Logistic Regression）

模型介紹：

- 羅吉斯迴歸是一種用於 處理分類 問題的 線性 模型。
- 通常用於解決 二元分類 問題，也可以擴展到多類別分類。
- 該模型 基於機率 的概念， 預測輸出屬於每個類別的機率 。

基本概念：

1. **二元分類**：將資料分為兩個類別，通常表示為0和1。
2. **模型輸出**：對於給定的輸入，模型會 輸出每個類別的機率 。
3. **機率相加為1**：模型輸出的各個類別機率相加為1，因此可以 將輸入歸屬於機率最高的類別 。

x --> nachine classfier --> y1, y2, ..., yk basic: how to classified? ex: 0,1 output: which catagories

貝氏定理應用：

- 用處：調整 後驗機率和先驗機率 之間的關係
  - 後驗機率（after having data）：p(g=k|X=x) 有x後，看你屬於第k組的機率
  - 先驗機率: 基於以往經驗的機率
- 方法：用貝氏定理來 計算後驗機率 ，即給定輸入資料後，資料屬於每個類別的機率。

訓練和測試：

- 訓練資料：
  - 用於估計模型的參數
  - 通常用 MLE 等方法來找到最佳參數
- 測試資料：
  - 評估模型 泛化能力 ，即模型對於未見過的資料的預測能力

表現不佳：

1. **調整參數估計的演算法**：試著調整模型參數估計的方法，這可能包括改變演算法的 停止條件 、增加或減少 迭代數量 等。
2. **更換演算法**
3. **資料調整**：進行資料的前處理，ex: 標準化
4. **特徵工程**：重新檢視特徵資料，可能需要增加、減少或者 改變特徵的選取方式 ，以提高模型對於資料的表現。

使用羅吉斯迴歸模型進行訓練和測試，並計算模型的準確率

- 概念：
  - 機率相等的地方在二維中是一條直線，三維中是一平面
  - 計算後驗機率, 誰大就分給誰
  - 用已經發生的資料讓他去學習, 讓估計 beta0 / beta1 估計好, 等新的資料進來去計算後驗機率
- 操作：
  - 讀取'Wine.xlsx'並轉換為 NumPy 陣列。
  - `train_test_split` ：資料集分為訓練(佔70%)和測試(佔30%)資料集
  - `StandardScaler` ：標準化處理資料，使其均值為 0，標準差為 1。

如何判斷哪些參數或方法是“最適合的”

- **交叉驗證（Cross-validation）**：
  - 使用交叉驗證來評估不同組合的性能。ex: k-fold 交叉驗證
  - 將數據分成 k 個子集，每次訓練模型時使用其中 k-1 個子集，並將剩餘的一個子集用於驗證

    - 重複進行 k 次，計算平均準確率或其他評估指標。
- **指標**：準確率、精確度、召回率、F1 分數
- **模型複雜**：模型的複雜度和泛化能（overfitting）
- **效率**：計算效率和訓練時間

參數調整

- 正規化
    - 防止模型過擬合的技術
    - 在損失函數中添加
    - 通常是通過 L1 正規化（Lasso 正規化）或 L2 正規化（Ridge 正規化）來實現
    - 控制正規化的強度:參數 `C` 越小，強度越大，可能會導致模型擬合程度較低，但對於泛化到未見過的數據可能更好。強度越高，越容易過度擬合
        - 訓練集表現良好+測試集表現差--過擬合；訓練集和測試集都不佳--欠擬合

In [ ]:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

df = pd.read_excel('Wine.xlsx')
X = np.array(df.iloc[:, :-1]) # Exclude the last column (label)
y = np.array(df.iloc[:, -1]) # Label column

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30)
scaler = StandardScaler()
X_train_ = scaler.fit_transform(X_train)
X_test_ = scaler.fit_transform(X_test)
```

ds0 -- lr

In [ ]:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

opts = dict(tol=1e-6, max_iter=int(1e6), verbose=0) # verbose=1 要不要列出中間過程
# solver = 'lbfgs'  # 'lbfgs' is the default
solver = 'liblinear'
# solver = 'newton-cg'
# 選擇一個演算法 solver
# opts: 如何停止我的演算法 跑過10**6次就該停
C_value = 0.1

# 初始化 LogisticRegression 模型，並設置 C 參數
clf_original = LogisticRegression(solver=solver, C=C_value, **opts)
# clf_original = LogisticRegression(solver=solver, **opts)
clf_original.fit(X_train_, y_train) # 開始訓練，要給資料 X_train
y_pred = clf_original.predict(X_test_) # 預測結果

# Report accuracy on test data 看預測結果好不好
# print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}\n")
print(f"Accuracy using clf_original.score: {clf_original.score(X_test_, y_test):.2%}\n")
# print(classification_report(y_test, y_pred))
```

```
Accuracy using clf_original.score: 98.15%
```

**學習曲線: 模型的性能如何隨著訓練數據量的增加而改變? **

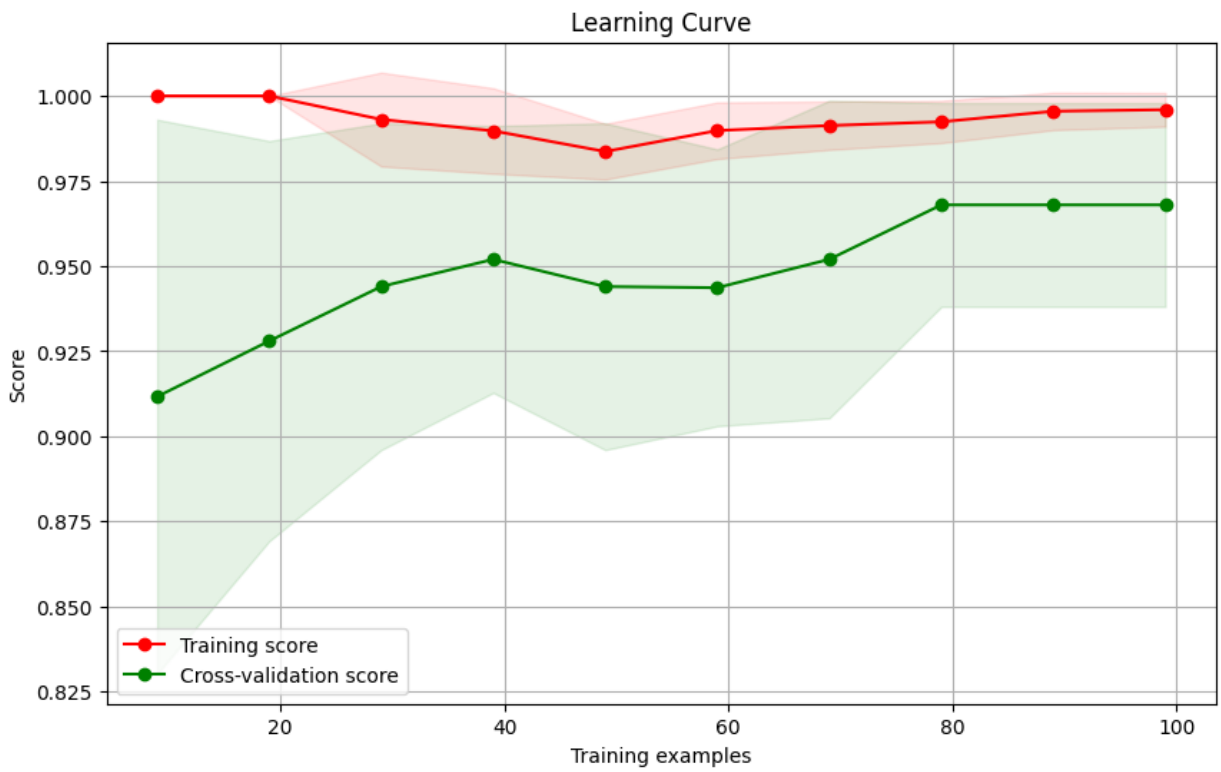
- 訓練分數（Training Score）：表示模型在看到更多訓練數據後是否能夠更好地擬合（fit）數據。
- 驗證分數（Validation Score）：表示模型對未見過的數據的泛化能力。
- 陰影的區域: 表示對於每個訓練數據量，分數的變化範圍

- 理想情況
    - 隨著訓練樣本數目的增加，訓練分數和驗證分數都應該趨近於收斂。
    - 訓練分數高於驗證分數，模型可能過度擬合的問題。

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import learning_curve

# Assume clf_originalCV is your MLPClassifier instance
# X_train and y_train are your training data

train_sizes, train_scores, valid_scores = learning_curve(
    clf_original, X_train_, y_train, train_sizes=np.linspace(0.1, 1.0, 10), cv=5)

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
valid_scores_std = np.std(valid_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, valid_scores_mean - valid_scores_std,
                 valid_scores_mean + valid_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
         label="Training score")
plt.plot(train_sizes, valid_scores_mean, 'o-', color="g",
         label="Cross-validation score")
```

```
plt.title("Learning Curve")
plt.xlabel("Training examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```



- param
  - solver = 'liblinear'
  - C_value = 0.1

輸出資訊介紹

1. Machine precision = 2.220D-16

- 機器精度為 2.220D-16
- 電腦能夠區分的最小數字的絕對值為 2.220×10的負16次方，這個數字以下的值會被電腦視為零

1. N = 42, M = 10

- N 表示參數數量為 42，M 表示限制條件的數量為 10

1. At X0, 0 variables are exactly at the bounds

- 在初始點 X0，有 0 個變數恰好在邊界上

1. At iterate 0, f = 1.36228D+02, |proj g| = 4.84876D+01

- 在第 0 次迭代時，函數值 f = 1.36228D+02，梯度的投影長度 |proj g| = 4.84876D+01

1. Accuracy using clf_original.score: 94.44%

- 使用 `clf_original.score` 計算得到的準確度為 94.44%

1. CONVERGENCE: REL_REDUCTION_OF*F*<=_FACTR*EPSMCH

- 收斂：相對函數減少量小於設定值 FACTR 乘以機器精度 EPSMCH

ds0 -- lr -- pca 將原始特徵資料ds0 / 用羅吉斯分類器 / 用主成分分析降維

In [ ]:

```
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# PCA transformation
pca = PCA(n_components=5).fit(X_train_)
Z_train = pca.transform(X_train_)
Z_test = pca.transform(X_test_)

opts = dict(tol=1e-6, max_iter=int(1e5), verbose=0)
# solver = 'lbfgs'  # 'lbfgs' is the default
solver = 'liblinear'
# solver = 'newton-cg'
C_value = 0.1

# 初始化 LogisticRegression 模型，並設置 C 參數
```

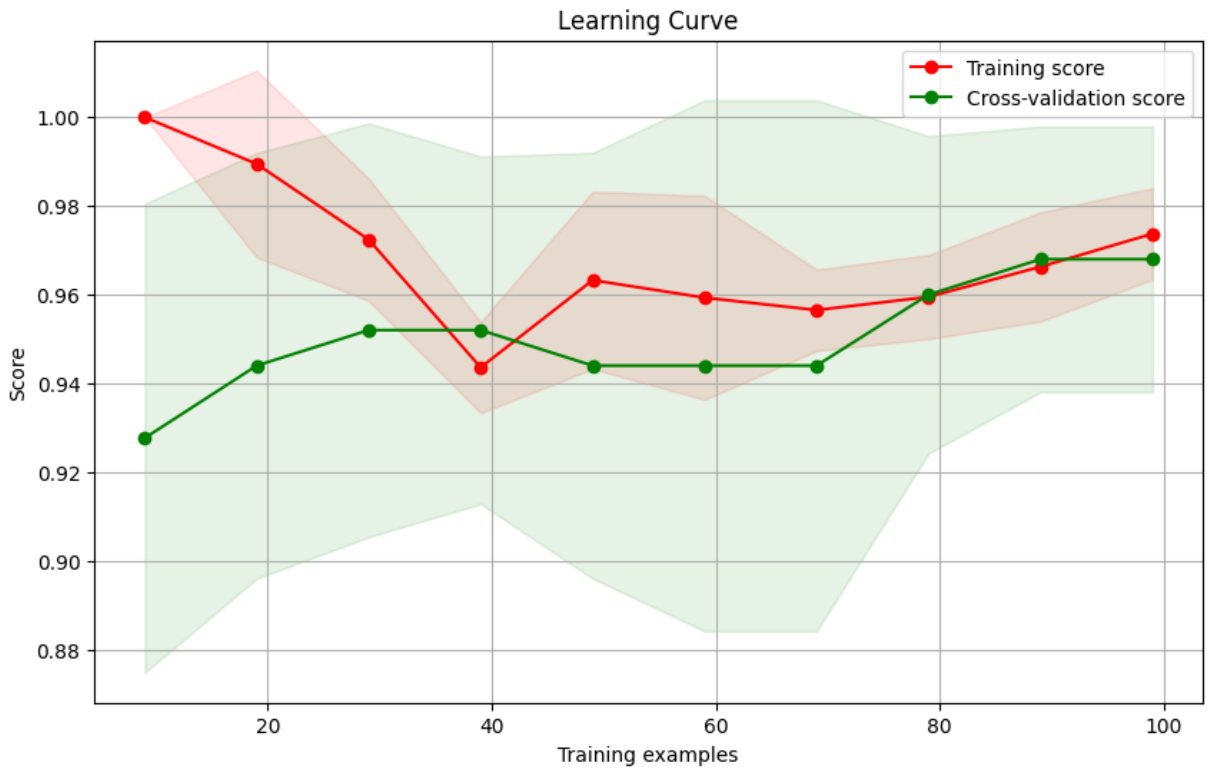
```
clf_PCA = LogisticRegression(solver=solver, C=C_value, **opts)
clf_PCA.fit(Z_train, y_train)
y_pred = clf_PCA.predict(Z_test)

# Report accuracy on test data
print(f"Accuracy using clf_PCA.score: {clf_PCA.score(Z_test, y_test):.2%}\n")
```

```
Accuracy using clf_PCA.score: 98.15%
```

1. 使用lbfgs演算法時，當我把c調成0.1，反而準確度比c=0.2還低
2. 使用liblinear演算法時，無論c是多少，準確度都一樣，於是我刪掉這個參數設定看看，發現準確度就降低了，所以持續使用這個參數。
3. 使用newton-cg演算法時，加入c反而讓準確度降低
4. pca降維後，最好的準確度是方案二下的94.44%，比沒有主成份分析的資料還要低，可推估是在pca過程時遺失一些資訊

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import learning_curve

# Assume clf_originalCV is your MLPClassifier instance
# X_train and y_train are your training data

train_sizes, train_scores, valid_scores = learning_curve(
    clf_PCA, Z_train, y_train, train_sizes=np.linspace(0.1, 1.0, 10), cv=5)

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
valid_scores_std = np.std(valid_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, valid_scores_mean - valid_scores_std,
                 valid_scores_mean + valid_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
         label="Training score")
plt.plot(train_sizes, valid_scores_mean, 'o-', color="g",
         label="Cross-validation score")

plt.title("Learning Curve")
plt.xlabel("Training examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```



支持向量機（SVM）

- **目標**：

1. 找到一個 最佳的超平面 ，將不同類別的 資料點分開 。
2. 迴歸分析中，則是尋找一條 最佳的迴歸線 ，以 最大化模型的預測能力 。 特色：能夠處理 線性和非線性 的問題，並具有很好的 泛化能力 ，對於 高維度 的資料集也表現良好。
3. 監督式學習演算法，用於 分類 和 迴歸分析 。
4. 超平面與最近數據點之間的距離。SVM的目標是最大化這個邊界 。

5. 非線性分類: SVM可以通過 `核函數` 有效地進行 `非線性` 分類，將輸入數據隱式映射到高維特徵空間。
6. 泛化能力：對於未見過的新資料的適應能力或預測能力，能夠避免過度擬合

- **數學公式**：
  - 超平面方程式：$\mathbf{w} \cdot \mathbf{x} + b = 0$，其中 $\mathbf{w}$ 是權重向量，$\mathbf{x}$ 是輸入向量，b 是偏差項。
- **分類**：
  - 二元: 分為兩個類別的最佳超平面
  - 多類別: 用一對一或一對多等策略
- **參數**：
  - RBF 核函數的核係數（Radial Basis Function）-- gamma
    - 控制每個數據點的影響範圍
    - 越小，相當於高斯核的 `標準差` 越大，支持向量的影響範圍越大，決策邊界越平滑
    - 較大，支持向量的影響範圍變小， `決策邊界更複雜` ， `更加貼合訓練數據`
  - poly -- degree
    - 多項式核函數 `將特徵空間映射到更高維度` 的空間，轉化非線性
    - 多項式核函數的次數，即將原始特徵空間映射到的高維空間的次數
    - `degree=3` ，映射到三次多項式特徵空間
- **演算法介紹**：
  - LinearSVC：適用線性可分的數據集，計算速度較快
  - SVC with RBF Kernel：使用高斯核函數（RBF），可以處理各種數據集，很強的擬合能力。對於過於簡單的問題，可能會過擬合
  - SVC with Polynomial Kernel：使用多項式核函數。處理非線性問題，但擬合力不如 RBF 核函數強

ds0 -- svm -- without std

In [ ]:

```
from sklearn.svm import SVC, LinearSVC
from sklearn.metrics import classification_report

C = 1  # SVM regularization parameter
opts = dict(C=C, tol=1e-6, max_iter=int(1e6))

# Choose one of the following classifiers by uncommenting the appropriate line
clf_svm = SVC(kernel="linear", **opts)
# clf_svm = SVC(kernel="rbf", gamma=0.2, **opts)
# clf_svm = SVC(kernel="poly", degree=2, gamma="auto", **opts)
# clf_svm = LinearSVC(**opts)  # one vs the rest

# Train the classifier
clf_svm.fit(X_train, y_train)

# Make predictions
predictions = clf_svm.predict(X_test)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           1       0.95      0.95      0.95        19
           2       0.95      0.95      0.95        20
           3       0.93      0.93      0.93        15

    accuracy                           0.94        54
   macro avg       0.94      0.94      0.94        54
weighted avg       0.94      0.94      0.94        54
```

1. 使用linear與poly, degree=2, gamma="auto"時，皆有0.94（方案中最高）的準確度.在poly中，我將degree下降1，成功讓準確度提升至方案中最高，不過再繼續下降degree便無法持續造成準確度的提升了
2. 使用rbf時，準確度都一樣極低，不到0.5
3. 唯一一個不使用SVC改用LinearSVC的方案確實也有不錯的準確度0.93，但還不是最好的

ds0 -- svm -- std

In [ ]:

```
from sklearn.svm import SVC, LinearSVC
from sklearn.metrics import classification_report

C = 0.1  # SVM regularization parameter
opts = dict(C=C, tol=1e-6, max_iter=int(1e6))

# Choose one of the following classifiers by uncommenting the appropriate line
clf_svm = SVC(kernel="linear", **opts)
# clf_svm = SVC(kernel="rbf", gamma=0.2, **opts)
# clf_svm = SVC(kernel="poly", degree=2, gamma="auto", **opts)
# clf_svm = LinearSVC(**opts)  # one vs the rest

# Train the classifier
clf_svm.fit(X_train_, y_train)

# Make predictions
predictions = clf_svm.predict(X_test_)

# Print classification report
print(classification_report(y_test, predictions))
```

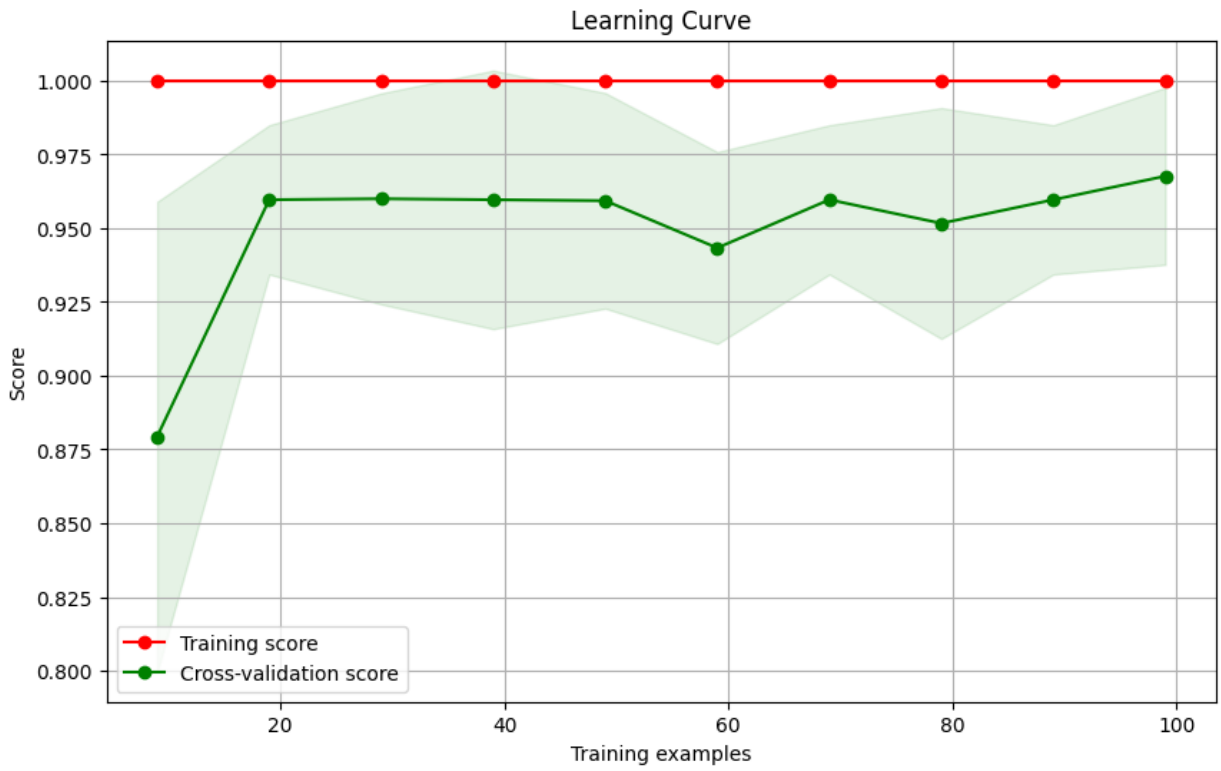
```
              precision    recall  f1-score   support
```

```
           1       1.00      1.00      1.00        19
           2       1.00      1.00      1.00        20
           3       1.00      1.00      1.00        15

    accuracy                           1.00        54
   macro avg       1.00      1.00      1.00        54
weighted avg       1.00      1.00      1.00        54
```

scale 資料後，準確度提升至100% 非常好

In [ ]:

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import learning_curve

# Assume clf_originalCV is your MLPClassifier instance
# X_train and y_train are your training data

train_sizes, train_scores, valid_scores = learning_curve(
    clf_svm, X_train_, y_train, train_sizes=np.linspace(0.1, 1.0, 10), cv=5)

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
valid_scores_std = np.std(valid_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, valid_scores_mean - valid_scores_std,
                 valid_scores_mean + valid_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
         label="Training score")
plt.plot(train_sizes, valid_scores_mean, 'o-', color="g",
         label="Cross-validation score")

plt.title("Learning Curve")
plt.xlabel("Training examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```



ds0 -- svm -- pca

In [ ]:

```python
from sklearn.svm import SVC, LinearSVC
from sklearn.metrics import classification_report

C = 0.1  # SVM regularization parameter
opts = dict(C=C, tol=1e-6, max_iter=int(1e6))

# Choose one of the following classifiers by uncommenting the appropriate line
# clf_svm = SVC(kernel="linear", **opts)
# clf_svm = SVC(kernel="rbf", gamma=0.1, **opts)
clf_svm = SVC(kernel="poly", degree=1, gamma="auto", **opts)
# clf_svm = LinearSVC(**opts)  # one vs the rest

# Train the classifier
clf_svm.fit(Z_train, y_train)
```

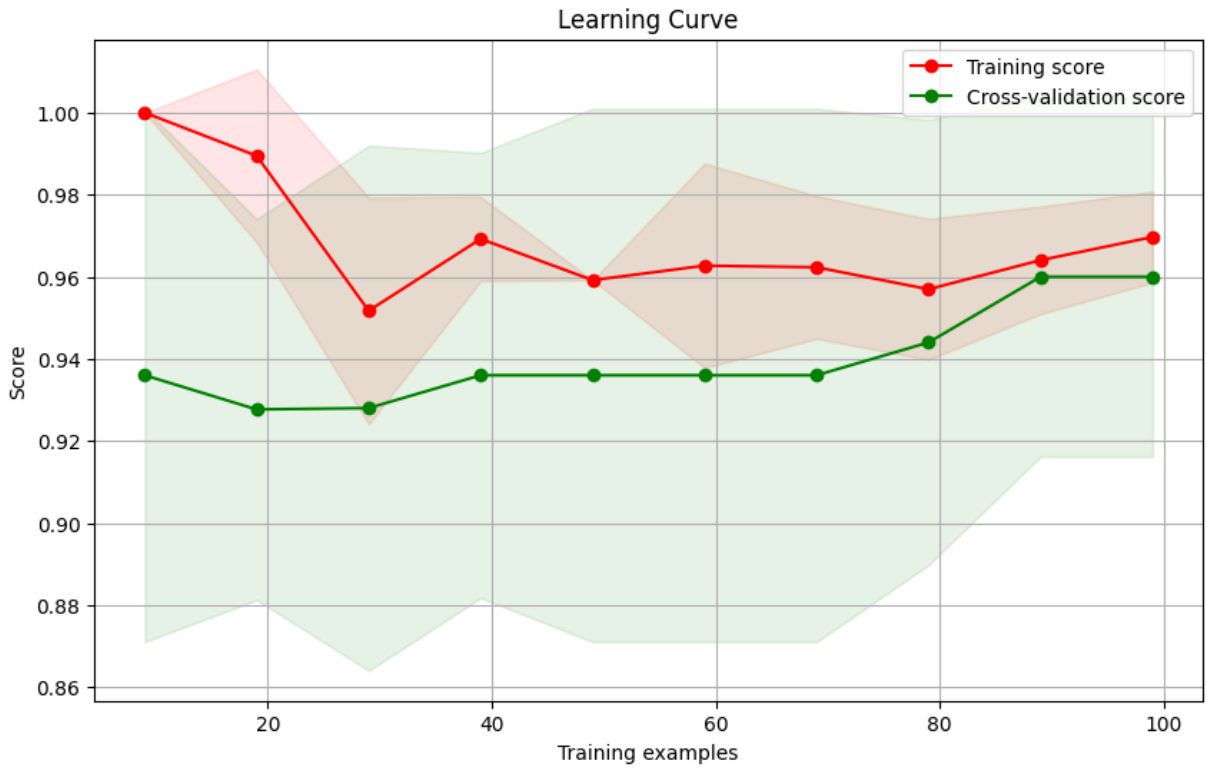
```
# Make predictions
predictions = clf_svm.predict(Z_test)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           1       1.00      1.00      1.00        19
           2       0.95      0.95      0.95        20
           3       0.93      0.93      0.93        15

    accuracy                           0.96        54
   macro avg       0.96      0.96      0.96        54
weighted avg       0.96      0.96      0.96        54
```

1. 使用poly, degree=1, gamma="auto" 以及 clf_svm = SVC(kernel="rbf", gamma=0.1, **opts) 時有0.96的準確度。degree=2(0.748)的準確度下降很多，甚至小於degree=3(0.89)，和沒有主成分分析預處理的資料相比，參數設定產生的結果很不一樣
2. 使用rbf時，準確度居然很好！相比沒有主成分分析預處理的資料，他的準確度提升很多
3. 唯一一個不使用SVC改用LinearSVC的方案確實也有不錯的準確度0.93，但還不是最好的

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import learning_curve

# Assume clf_originalCV is your MLPClassifier instance
# X_train and y_train are your training data

train_sizes, train_scores, valid_scores = learning_curve(
    clf_svm, Z_train, y_train, train_sizes=np.linspace(0.1, 1.0, 10), cv=5)

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
valid_scores_std = np.std(valid_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, valid_scores_mean - valid_scores_std,
                 valid_scores_mean + valid_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
         label="Training score")
plt.plot(train_sizes, valid_scores_mean, 'o-', color="g",
         label="Cross-validation score")

plt.title("Learning Curve")
plt.xlabel("Training examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```



神經網路(Neural Network) -- 具備隱藏層的前饋式(feedforward)多層次感知器神經網路

- **數學模型**：

- 前饋式神經網路的數學關係式：激活函數、注意輸出輸入
  - input(layer) => NN learner => output(layer) y(y is catogories)
  - 輸入輸出之間有函數關係
- 激活函數（activation function）：加入非線性，模擬神經細胞的激發狀態。
  - 常見的激活函數（sigmoid）：[g(z) = \frac{1}{1 + e^{-z}}]
- 損失函數（loss function）：衡量實際與預測的輸出之間相差多少，並嘗試最小化（類似LSE的概念）
  - MSE 和 交叉熵損失（Cross-Entropy Loss）

- **模型訓練**：
  - 最小化損失函數: 調整 `權重` 和 `偏差` ，使實際與預測輸出相差最小化
  - MLPRegressor / MLPClassifier: 於處理輸出為連續型 / 類別型變數

1. **權重（weights）**：
   - represent: 不同輸入對神經元的影響程度
   - relation: 權重增減，使神經元 `對不同輸入的敏感度` 增加或減少
   - purpose: 訓練過程神經網路會自動調整權重，以 `使得輸出更接近預期輸出`
2. **偏差（biases）**：
   - represent: 神經元被激活的容易程度
   - purpose: 使神經元`對不同輸入表現不同的激活程度``

多層感知器（MLP）

- **數學公式**：
  - MLP是一種具有多層節點（神經元）的人工神經網絡，使用非線性激活函數。
  - 網絡中的每個神經元計算其輸入的加權和，並通過激活函數產生輸出。
- **關鍵概念**：
  - **激活函數**：應用於輸入加權和的函數，引入網絡的非線性，使其能夠學習複雜的模式。
    - **ReLU（Rectified Linear Unit）**：
    - 定義：f(x) = max(0, x)。
    - 特點：
      - 較好性能和速度
      - 解決梯度消失: 保留正數輸入，負數輸入轉為零，具稀疏性
    - **Logistic 函數**：
    - 定義：f(x) = 1 / (1 + exp(-x))。
    - 特點：
      - sigmoid 函數
      - 將輸入值映射到 (0, 1) 的區間內，用於 `二元分類` 問題
      - 有平滑的 S 形曲線，良好微分性質
    - **Tanh 函數**：
      - 定義：f(x) = (exp(x) - exp(-x)) / (exp(x) + exp(-x))。
      - 特點：
        - 將輸入值映射到 (-1, 1) 的區間內。
        - `零中心化` 特性， `平均值為 0` ，有助於收斂速度。
  - **反向傳播**：一種訓練算法，通過將錯誤反向傳播從輸出層向輸入層，更新網絡的權重。

- **分類**：
  - MLP可用於二元分類和多類別分類任務。
  - 它是一種通用函數逼近器，這意味著只要具有足夠的隱藏單元和訓練數據，它就可以學習逼近任何函數。
  - MLP使用優化算法（例如梯度下降）訓練，以最小化損失函數，對於分類任務通常使用交叉熵損失。

ds0 -- nn

In [ ]:

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

# Set the parameters for the MLPClassifier
hidden_layers = (48,)  # one hidden layer with 30 units
activation = 'logistic'
# alpha = 0.2  # regularization parameter
# beta_1 = 0.9  # exponential decay rate for estimates of first moment vector in adam
opts = dict(hidden_layer_sizes=hidden_layers, verbose=False, activation=activation, tol=1e-6, max_iter=1000,
        )

solver = 'adam'  # default solver

# Create the MLPClassifier
clf_MLP = MLPClassifier(solver=solver, **opts)

# Train the classifier
clf_MLP.fit(X_train_, y_train)

# Make predictions
```

```
predictions = clf_MLP.predict(X_test_)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           1       1.00      1.00      1.00        19
           2       0.95      1.00      0.98        20
           3       1.00      0.93      0.97        15

    accuracy                           0.98        54
   macro avg       0.98      0.98      0.98        54
weighted avg       0.98      0.98      0.98        54
```

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
```

- 我將隱藏層設定有48個神經元，因為我讓神經元在50-55個之間跑很多次，發現48是最穩定的擁有最高準確度的設定，大部分準確度至少有0.94，偶爾可以達到0.96
- 相對其他演算法，adam表現最優秀
- 激活函數也以logistic表現最好，tanh 表現也不錯但要跑比較久，而relu在這則需要足夠多的神經元才能達到好性能
- alpha = 0.2
- beta_1 = 0.9

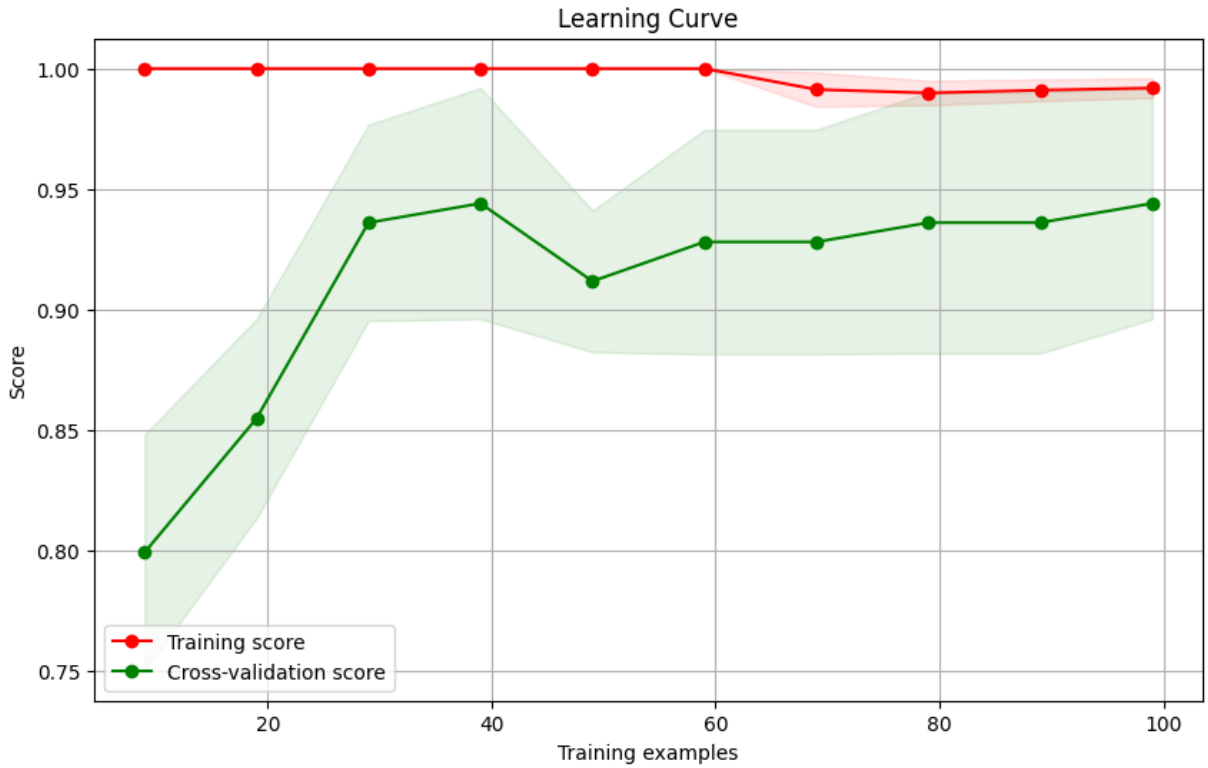
without std

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import learning_curve

# Assume clf_MLP is your MLPClassifier instance
# X_train and y_train are your training data

train_sizes, train_scores, valid_scores = learning_curve(
    clf_MLP, X_train, y_train, train_sizes=np.linspace(0.1, 1.0, 10), cv=5)

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
valid_scores_std = np.std(valid_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, valid_scores_mean - valid_scores_std,
                 valid_scores_mean + valid_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
         label="Training score")
plt.plot(train_sizes, valid_scores_mean, 'o-', color="g",
         label="Cross-validation score")

plt.title("Learning Curve")
plt.xlabel("Training examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
```

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
```



std

ds0 -- nn -- pca

In [ ]:

```python
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

# Set the parameters for the MLPClassifier
hidden_layers = (72,)  # one hidden layer with 30 units
activation = 'logistic'
# alpha = 0.1  # regularization parameter
# beta_1 = 0.9  # exponential decay rate for estimates of first moment vector in adam
opts = dict(hidden_layer_sizes=hidden_layers, verbose=False, activation=activation, tol=1e-6, max_iter=1000,
        )
solver = 'sgd'  # default solver

# Create the MLPClassifier
clf_MLP = MLPClassifier(solver=solver, **opts)

# Train the classifier
clf_MLP.fit(Z_train, y_train)

# Make predictions
predictions = clf_MLP.predict(Z_test)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           1       1.00      1.00      1.00        19
           2       1.00      0.95      0.97        20
           3       0.94      1.00      0.97        15

    accuracy                           0.98        54
   macro avg       0.98      0.98      0.98        54
weighted avg       0.98      0.98      0.98        54
```

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
```

```
  warnings.warn(
```

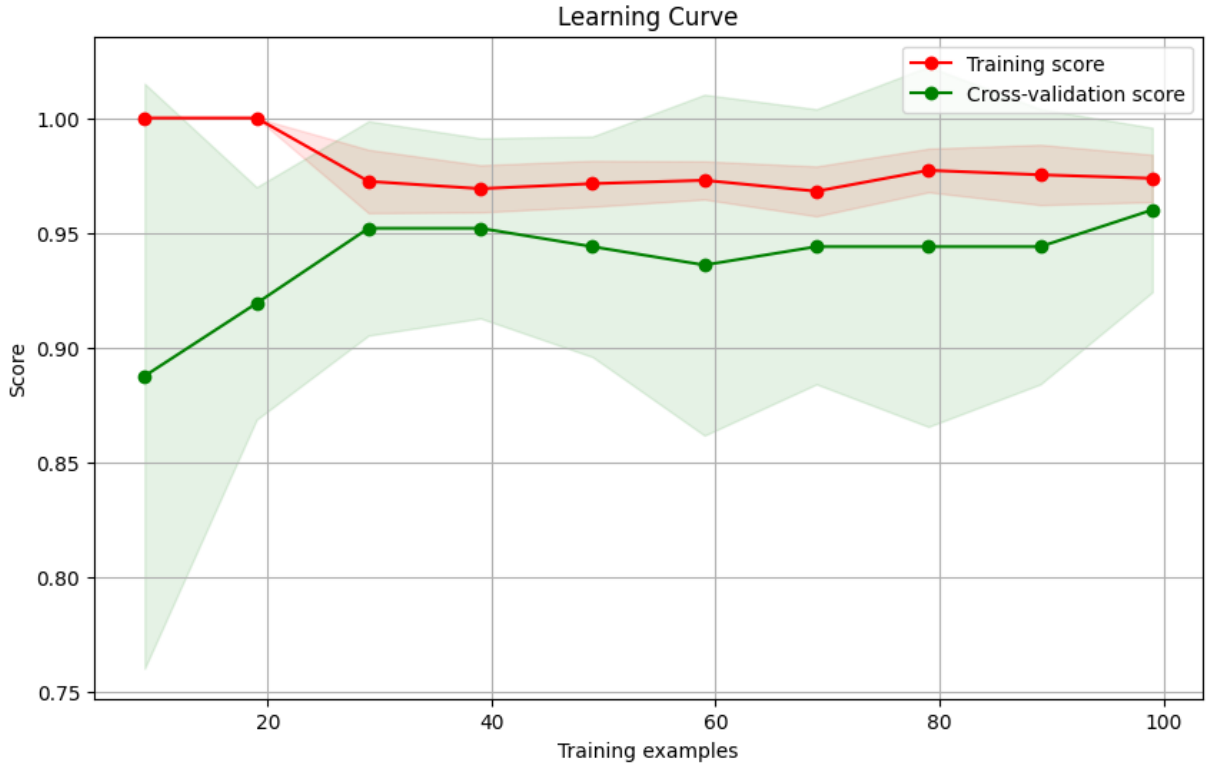
- pca後的資料，就算大範圍（try 20-130）的調整隱藏層神經元數量，準確度皆為0.93
- beta1 = 0.9 是最好的參數設定，再高或再低都會降低準確度（即使只是提高到0.98）
- 而演算法無論選擇adam or sgd or lbfgs都沒什麼差異

In [ ]:

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import learning_curve

# Assume clf_originalCV is your MLPClassifier instance
# X_train and y_train are your training data

train_sizes, train_scores, valid_scores = learning_curve(
    clf_MLP, Z_train, y_train, train_sizes=np.linspace(0.1, 1.0, 10), cv=5)

train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
valid_scores_std = np.std(valid_scores, axis=1)

plt.figure(figsize=(10, 6))
plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                 train_scores_mean + train_scores_std, alpha=0.1, color="r")
plt.fill_between(train_sizes, valid_scores_mean - valid_scores_std,
                 valid_scores_mean + valid_scores_std, alpha=0.1, color="g")
plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
         label="Training score")
plt.plot(train_sizes, valid_scores_mean, 'o-', color="g",
         label="Cross-validation score")

plt.title("Learning Curve")
plt.xlabel("Training examples")
plt.ylabel("Score")
plt.legend(loc="best")
plt.grid(True)
plt.show()
```

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
```

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multilayer_perceptron.p
y:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(
```



最佳組合計算 -- 最好的參數和對應的演算法

1. 匯入所需的庫：
   - `GridSearchCV` ：用於在給定參數網格中執行 交叉驗證的超參數搜索 。
   - `StratifiedShuffleSplit` ：用於創建分層隨機拆分，確保訓練集和測試集中類的分佈相似。
   - `datetime` ：用於獲取當前日期和時間。
2. 獲取當前日期和時間，並將其格式化為字符串表示形式，以便用於結果文件的命名。
3. 定義參數：
   - `opts` ：
     - 收斂: 參數只要夠好，代表優化已達到最佳，計算就會停止，專注在模型是否穩定
     - 最大迭代次數: 允許算法執行的最多次數，避免無止盡計算
   - `parameters` ：包括邏輯回歸模型的求解器和正則化強度C的值。
4. 使用 `StratifiedShuffleSplit` 創建了一個分層的隨機拆分交叉驗證對象 `cv` ，用於後續的網格搜索中。
5. 初始化了 `GridSearchCV` 對象 `grid` ，傳入了以下參數：
   - `estimator` ：邏輯回歸模型的初始化器，使用了 `opts` 中的參數。
   - `param_grid` ：要搜索的參數網格，包括邏輯回歸的求解器和正則化強度C的值。
   - `cv` ：交叉驗證策略。
   - `scoring` ：性能評估指標，包括準確率和宏平均F1值。
   - `refit` ：在找到最佳參數後，將模型重擬合到整個訓練集以便評估。
6. 使用 `fit` 方法對訓練數據 `X_train` 和標籤 `y_train` 進行網格搜索，以找到最佳參數組合。
7. 將交叉驗證結果保存到CSV文件中，並打印出最佳參數、最佳得分和最佳估計器。

In [ ]:

```
# from sklearn.model_selection import GridSearchCV, StratifiedShuffleSplit
# from datetime import datetime

# # Get the current date and time
# now = datetime.now()

# # Format the date and time as a string
# now_str = now.strftime("%Y_%m_%d_%H_%M_%S")
# results_file = 'data/results_' + now_str + '.csv'

# opts = dict(tol=1e-6, max_iter=int(1e6))  # parameters for LogisticRegression
# parameters = {'solver': ['lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga'], 'C': [0.1, 1, 10]}  # parameters for GridSearchCV

# # parameters = {'solver':['lbfgs', 'liblinear', 'newton-cg', 'sag','saga']}

# cv = StratifiedShuffleSplit(n_splits=5, test_size=0.3, random_state=0)  # 5-fold CV

# grid = GridSearchCV(estimator=LogisticRegression(**opts), param_grid=parameters, cv=cv,
#                     scoring=['accuracy', 'f1_macro'], refit="accuracy")

# grid.fit(X_train, y_train)
# # grid.fit(X, y)

# cv_logistic = pd.DataFrame(data=grid.cv_results_)
# cv_logistic.to_csv(results_file)

# print(grid.best_params_)
# print(grid.best_score_)
# print(grid.best_estimator_)
```

1. 計算模型準確度：

- `score = 100 * clf_MLP.score(X_test, y_test)` ：計算多層感知器（MLP）模型在測試數據上的準確度。
2. 顯示混淆矩陣：
   - `disp = ConfusionMatrixDisplay.from_estimator(clf_MLP, X_test, y_test, xticks_rotation=45, cmap=plt.cm.Blues, normalize='true', ax=ax)` ：使用 `ConfusionMatrixDisplay.from_estimator()` 參數：
     - `clf_MLP` ：已經訓練好的多層感知器模型。
     - `cmap=plt.cm.Blues` ：使用藍色調色板來顯示混淆矩陣。
     - `normalize='true'` ：對混淆矩陣進行正規化，以顯示每個類別的相對比例
3. 概念：
   - 混淆矩陣通過 `總結每個類別的正確和錯誤預測數量` 來提供對模型性能的洞察
   - 對角線元素: 代表正確分類的實例，而對角線之外的元素表示錯誤分類。
   - 0.86 表示有 86 個實例被正確分類為屬於類別 2
   - 類別 1、2 和 3 代表分類器做出的預測類別分配，其解釋取決於原始數據中定義的類別標籤或類別

結論：從以上評比實驗可知

- 羅吉斯迴歸（Logistic Regression）
  - 標準化資料：98.51%
  - PCA資料：96.30%
  - 交叉驗證：94.44%
- 支援向量機（Support Vector Machine, SVM）
  - 未標準化資料：94%
  - 標準化資料：100%
  - PCA資料：96%
- 神經網路（Multi-layer Perceptron, MLP）
  - 標準化資料：98%
  - PCA資料：98%

- 資料是否經過主成份分析真的有不小的影響，同樣的演算法或參數變化，能對標準化資料和PCA資料有相反的影響，所以不能同一個分類器、同一個資料就不去換參數。由此可知，資料如何預處理對後續的分析也會有很大的影響。

-**比較**

1. 運用羅吉斯分類器，我將參數設定opts = dict(tol=1e-6, max_iter=int(1e6), verbose=0)，並採liblinear演算法，將正規化參數設為0.1，產出98.15%的最佳準確度。 主成分取前5個主成分（原因：取到第五個之後，就算多取幾個準確度也沒什麼變化），我們也用相同設定並獲得96.3%的準確度。而兩個資料的速度都跑得很快，沒明顯的差異，而三種演算法跑出的準確度也差不多，不過newton-cg會花上較久的時間。 再來，我們看學習曲線發現，標準化資料比主成分資料穩定（理想）許多，標準化資料的泛化能力有隨著樣本數增加，而訓練能力無論樣本大小都很好，但主成分資料的兩者皆不太穩定也不夠理想，但最後兩者有趨近。

2. 在svm分類器中，我皆選用linear演算法，並在為標準化的原始資料中將正規化參數設為1，因為1或0.1都不影響到準確度變化。 反之，標準化後的原始資料將c=0.1能獲得高達100%的準確度，是非常好的表現，且運作速度也很快。從學習曲線來看，標準化資料的訓練分屬恆維持在1，而泛化能力也表現很好。 在主成分在上面我採用不同的演算法SVC(kernel="poly", degree=1, gamma="auto", **opts)並且調整他的degree，讓他獲得96%的準確度，比liblinear演算法表現的還要好，而使用clf_svm = SVC(kernel="rbf", gamma=0.1,** opts)也能獲得一樣的效果。

3. 在神經網路中，我將原始資料與pca資料選用相同的激活函數logistic，將隱藏層神經元數量一個設48，一個設72，不同的演算法，一個是adam，一個是sgd，而兩者最終都獲得98%的準確，但由於pca的神經元比較多（72，再多加上去準確度都不大影響），因此花費了較多的時間去跑。 從學習曲線來看，主成分資料最後紅線和綠線十分逼近，應該能避免過度擬合的問題，不過訓練分數稍稍比原始資料不穩定些。

建議

1. 如果希望準確度越高越好，那最適合讓標準化原始資料在svm分類器底下進行分類，其最佳組合為 C = 0.1；opts = dict(C=C, tol=1e-6, max_iter=int(1e6))；clf_svm = SVC(kernel="linear", **opts)，而他花的時間也很短。
2. 如果希望在主成分資料中獲得較高的準確度，那可以選擇神經網路分類器，將其設定為hidden_layers = (72,) ； activation = 'logistic'; solver = 'sgd'
3. 從上述可見，標準化的資料準確度通常比主成分來的更好更穩定，不過我們只取前五個主成分，應該也減少了不少資料量，各有各的好處。如果希望有一個分類器能在兩種資料上都表現一樣優良，那可以選擇神經元網路，其餘分類器都偏向標準化資料表現較優。而羅吉斯分類器表現最不好，因此在這邊不會想選用他。